Atividades práticas de Computação Gráfica

Curso de Ciência da Computação
Professor Marcelo R. Thielo

1. Filtro de miniatura com blur externo e sharpen central

Implemente em C uma função que receba `struct image *in` (imagem PPM RGB) e retorne uma **nova** `struct image *` aplicando um efeito “miniatura” onde as bordas da imagem ficam progressivamente desfocadas e o centro fica mais nítido, conforme visto nos slides de aula.

- **Sugestão de Assinatura**:

```
None
struct image *filtro_miniatura(struct image *in);
```

- Aplique um **Gaussian blur** com o kernel 3x3. Para obter um grau maior ou menor de blur, repita a operação quantas vezes forem necessárias.
- Implemente **unsharp mask** ou um filtro de nitidez simples (por exemplo: output = original + α·(original − blurred)), onde α depende da distância do centro. Aplique o sharpen preferencialmente ao centro vertical.
- **P**ara cada pixel combine os resultados de maneira ponderada pelo mapa vertical: $out(x,y)=wblur(y)\cdot blur(x,y)+wsharp(y)\cdot sharp(x,y)$.
- Processe cada canal separadamente; mantenha valores no intervalo válido [0,255] com saturação adequada.
- Escolha e documente uma estratégia para as bordas da imagem(zero padding, replicação ou reflexão).
- Retorne uma nova imagem alocada dinamicamente; evite cópias desnecessárias.
- Pense em otimizações.

2. Detecção de bordas com Laplaciano e Sobel

Implemente, em C, um programa que aplique quatro métodos de detecção de bordas a imagens PPM RGB , **Sobel (magnitude total)**, **Sobel Horizontal**, **Sobel Vertical** e **Laplaciano** , e compare os resultados visualmente e por métricas simples. Como bônus, incluir um módulo de **detecção de movimento** entre dois frames usando diferenças nas bordas.

Requisitos mínimos

- **Entrada**: uma imagem PPM RGB (para comparação de métodos) e, para o bônus, dois frames PPM consecutivos (mesma resolução).

- **Saída**: quatro imagens PPM em tons de cinza (R=G=B) com as bordas detectadas por cada método; para o bônus, uma imagem/relatório indicando regiões de movimento.

- **Assinaturas sugeridas para as funções**:

None

```
- struct image *edges_sobel(struct image *in);        // magnitude Sobel
- struct image *edges_sobel_h(struct image *in);      // Sobel horizontal
- struct image *edges_sobel_v(struct image *in);      // Sobel vertical
- struct image *edges_laplacian(struct image *in);    // Laplaciano 3x3
- struct image *motion_from_edges(struct image *A, struct image *B); // bônus
```

Processamento: converter para luminância antes de aplicar filtros; normalizar e mapear resultados para [0,255].

Implementação e passos sugeridos

- **Conversão para luminância**: I=0.299R+0.587G+0.114B.
- **Suavização opcional**: aplicar um blur gaussiano leve (σ≈1) para reduzir ruído antes do gradiente.
- **Sobel**:
    - Calcule Gx e Gy com kernels 3×3 padrão (bordas horiz e verticais).
    - **Magnitude total**: M=Gx2+Gy2 → normalize para saída.
    - **Horizontal**: use ∣Gx∣ ou Gx mapeado para tons de cinza.

    - **Vertical**: use $|Gy|$ ou Gy mapeado para tons de cinza.

  Laplaciano:

    - Use um kernel 3×3
    - Tome valor absoluto ou realce de cruzamento de zeros; normalize para saída.
- **Comparação**:
    - Gere as quatro imagens de borda lado a lado (ou salve separadas).
    - Calcule métricas simples por método: **média da magnitude**, **número de pixels acima de limiar** e **razão sinal/ruído local** (opcional).
    - Produza um breve comentário (texto) sobre diferenças visuais e métricas.

Bônus: Detecção de Movimento por Bordas

- **Entrada**: frames A e B consecutivos.
- **Método simples**:
    - Calcule mapa de bordas (por exemplo, Sobel magnitude) para ambos: EA, EB.
    - Calcule diferença absoluta $D = |EB - EA|$.
    - Aplique limiar adaptativo em D para obter máscara binária de movimento.
    - Opcional: aplicar morfologia (erosão/dilatação) para remover ruído e preencher regiões.
- **Saída**: imagem em tons de cinza com intensidade proporcional à mudança de borda e uma máscara binária de movimento; relatório com área total em movimento (número de pixels).
- **Aplicação prática**: detectar deslocamentos de objetos, mudanças de cena ou movimento da câmera; documentar limitações (sensível a iluminação e ruído).

Entregáveis

- Código-fonte em C com `Makefile` ou instruções de compilação.
- Programa de teste que lê PPM(s), gera as imagens de borda e, no bônus, a máscara de movimento, e imprime métricas básicas.
- README curto explicando parâmetros, escolhas e observações sobre as diferenças entre métodos.